

Universidad Tecnológica de Panamá

Facultad de Ingeniería en Sistema Computacionales

Licenciatura de Ciberseguridad

Profesor

Napoleon Ibarra

Estudiante

Stefany Guerrero

Materia

Programación II

Fecha

10/4/2026

Semestre I

menu.py

```
import tkinter as tk
from tkinter import messagebox
import subprocess
import sys
import os
```

```
def centrar_ventana(ventana, ancho, alto):
    ventana.update_idletasks()
    x = (ventana.winfo_screenwidth() - ancho) // 2
    y = (ventana.winfo_screenheight() - alto) // 2
    ventana.geometry(f"{ancho}x{alto}+{x}+{y}")
```

```
def ejecutar_problema1():
    try:
        subprocess.Popen([sys.executable, "problema1.py"])
    except Exception as e:
        messagebox.showerror("Error", f"No se pudo ejecutar Problema 1: {e}")
```

```
def ejecutar_problema2():
    try:
        subprocess.Popen([sys.executable, "problema2.py"])
    except Exception as e:
        messagebox.showerror("Error", f"No se pudo ejecutar Problema 2: {e}")
```

```
def salir():
    root.destroy()
```

```
# Ventana principal
root = tk.Tk()
root.title("Taller Práctico - Programación II")
centrar_ventana(root, 450, 350)
root.configure(bg="#F0F0F0")
root.resizable(False, False)
```

```
# Título
tk.Label(root, text="Taller Práctico", font=("Arial", 16, "bold"),
```

```
bg="#F0F0F0").pack(pady=30)
tk.Label(root, text="Seleccione un problema:", font=("Arial", 11),
bg="#F0F0F0").pack(pady=10)

# Botones
tk.Button(root, text="Problema 1 - Gestión de Notas",
command=ejecutar_problema1,
        bg="#0078D7", fg="white", font=("Arial", 11), padx=20, pady=10,
width=30).pack(pady=8)

tk.Button(root, text="Problema 2 - Encriptación de Archivos",
command=ejecutar_problema2,
        bg="#9C27B0", fg="white", font=("Arial", 11), padx=20, pady=10,
width=30).pack(pady=8)

tk.Button(root, text="Salir", command=salir,
        bg="#D32F2F", fg="white", font=("Arial", 11), padx=20, pady=10,
width=30).pack(pady=20)

root.mainloop()
```

Problema 1

```
import tkinter as tk
from tkinter import ttk, messagebox
import json
import os
import re

# ===== CONFIGURACIÓN =====
COLOR_FONDO = "#F0F0F0"
COLOR_BLANCO = "#FFFFFF"
COLOR_AZUL = "#0078D7"
COLOR_ROJO = "#D32F2F"
COLOR_VERDE = "#388E3C"

ARCHIVO = "respaldo_notas.json"
datos = {"profesores": []}

# ===== FUNCIONES REUTILIZABLES =====
def centrar(ventana, ancho, alto):
    try:
        ventana.update_idletasks()
        x = (ventana.winfo_screenwidth() - ancho) // 2
        y = (ventana.winfo_screenheight() - alto) // 2
        ventana.geometry(f"{ancho}x{alto}+{x}+{y}")
    except:
        pass

def guardar():
    try:
        with open(ARCHIVO, "w", encoding="utf-8") as f:
            json.dump(datos, f, indent=4, ensure_ascii=False)
            messagebox.showinfo("Éxito", "Datos guardados")
    except Exception as e:
        messagebox.showerror("Error", f"No se guardó: {e}")
```

```
def cargar():
    global datos
    try:
        if os.path.exists(ARCHIVO):
            with open(ARCHIVO, "r", encoding="utf-8") as f:
                datos = json.load(f)
            actualizar_vista()
            actualizar_combos()
    except Exception as e:
        messagebox.showerror("Error", f"No se cargó: {e}")
```

===== VALIDACIONES =====

```
def solo_letras_y_acentos(texto):
    try:
        if not texto:
            return False
        patron = r'^[a-zA-ZáéíóúÁÉÍÓÚñÑçÇ\s]+$'
        return re.match(patron, texto) is not None
    except:
        return False
```

```
def solo_letras_numeros_espacios(texto):
    try:
        if not texto:
            return False
        patron = r'^[a-zA-ZáéíóúÁÉÍÓÚñÑçÇ0-9\s]+$'
        return re.match(patron, texto) is not None
    except:
        return False
```

```
def validar_nombre(nombre, tipo):
    try:
        if not nombre or nombre.strip() == "":
            return " ⚠ Escriba un nombre"
```

```

nombre = nombre.strip()

if len(nombre) > 30:
    return f" ⚠ Máximo 30 caracteres (tiene {len(nombre)})"

if tipo == "grupo":
    if not solo_letras_numeros_espacios(nombre):
        return " ⚠ Solo letras, números y espacios"
    else:
        if not solo_letras_y_acentos(nombre):
            return " ⚠ Solo letras, espacios y acentos (sin números)"
        return None
except:
    return " ⚠ Error en validación"

def solo_numeros(texto):
    try:
        if texto in ["", "-", "."]:
            return True
        if texto.count('.') > 1:
            return False
        for c in texto:
            if c not in "0123456789.-":
                return False
        valor = float(texto)
        if valor < 0 or valor > 100:
            return False
        if '.' in texto:
            decimales = texto.split('.')[1]
            if len(decimales) > 2:
                return False
        return True
    except:
        return False

```

```
def limpiar(*entries):
    try:
        for e in entries:
            e.delete(0, tk.END)
    except:
        pass
```

```
def limitar_30(texto):
    return len(texto) <= 30
```

```
def actualizar_vista():
    try:
        lista.delete(0, tk.END)
        if not datos.get("profesores"):
            lista.insert(tk.END, "📁 No hay profesores registrados")
            return
        for p in datos["profesores"]:
            lista.insert(tk.END, f"👤 {p['nombre']}")
            if not p.get("asignaturas"):
                lista.insert(tk.END, f"📁 Sin asignaturas")
                continue
            for a in p["asignaturas"]:
                lista.insert(tk.END, f"📖 {a['nombre']}")
                if not a.get("grupos"):
                    lista.insert(tk.END, f"📁 Sin grupos")
                    continue
                for g in a["grupos"]:
                    icono = "🏠" if g["modalidad"] == "Presencial" else "💻"
                    lista.insert(tk.END, f" {icono} {g['nombre']}")
                    if not g.get("estudiantes"):
                        lista.insert(tk.END, f"📁 Sin estudiantes")
                    else:
                        for e in g["estudiantes"]:
                            lista.insert(tk.END, f"🎓 {e['nombre']}")
    except Exception as e:
```

```
messagebox.showerror("Error", f"Error al actualizar vista: {e}")
```

```
def actualizar_combos():
```

```
    try:
```

```
        profs = [p["nombre"] for p in datos.get("profesores", [])]
```

```
        combo_prof["values"] = profs
```

```
        combo_prof.set(profs[0] if profs else "")
```

```
        actualizar_combo_asig()
```

```
    except:
```

```
        pass
```

```
def actualizar_combo_asig():
```

```
    try:
```

```
        prof = combo_prof.get()
```

```
        for p in datos.get("profesores", []):
```

```
            if p["nombre"] == prof:
```

```
                asigs = [a["nombre"] for a in p.get("asignaturas", [])]
```

```
                combo_asig["values"] = asigs
```

```
                combo_asig.set(asigs[0] if asigs else "")
```

```
                actualizar_combo_grupo()
```

```
                break
```

```
    except:
```

```
        pass
```

```
def actualizar_combo_grupo():
```

```
    try:
```

```
        prof, asig = combo_prof.get(), combo_asig.get()
```

```
        for p in datos.get("profesores", []):
```

```
            if p["nombre"] == prof:
```

```
                for a in p.get("asignaturas", []):
```

```
                    if a["nombre"] == asig:
```

```
                        grupos = [g["nombre"] for g in a.get("grupos", [])]
```

```
                        combo_grupo["values"] = grupos
```

```
                        combo_grupo.set(grupos[0] if grupos else "")
```

```
                        return
```



```
except:  
    pass
```

```
def obtener_grupo():  
    try:  
        prof, asig, gru = combo_prof.get(), combo_asig.get(), combo_grupo.get()  
        if not prof or not asig or not gru:  
            return None  
        for p in datos["profesores"]:  
            if p["nombre"] == prof:  
                for a in p["asignaturas"]:  
                    if a["nombre"] == asig:  
                        for g in a["grupos"]:  
                            if g["nombre"] == gru:  
                                return g  
        return None  
    except:  
        return None
```

```
def agregar(entry, tipo):  
    try:  
        nombre = entry.get().strip()  
        error = validar_nombre(nombre, tipo)  
        if error:  
            messagebox.showwarning("Atención", error)  
            return  
  
        prof = combo_prof.get() if tipo != "profesor" else None  
  
        if tipo != "profesor" and not prof:  
            messagebox.showwarning("Atención", "Seleccione un profesor")  
            return  
  
        agregado = False  
  
        if tipo == "profesor":
```

```

for p in datos["profesores"]:
    if p["nombre"].lower() == nombre.lower():
        messagebox.showwarning("Atención", "Profesor ya existe")
        return
datos["profesores"].append({"nombre": nombre, "asignaturas": []})
agregado = True

elif tipo == "asignatura":
    for p in datos["profesores"]:
        if p["nombre"] == prof:
            for a in p["asignaturas"]:
                if a["nombre"].lower() == nombre.lower():
                    messagebox.showwarning("Atención", "Asignatura ya existe")
                    return
            p["asignaturas"].append({"nombre": nombre, "grupos": []})
            agregado = True
            break

elif tipo == "grupo":
    modalidad = combo_mod.get()
    if not modalidad:
        messagebox.showwarning("Atención", "Seleccione modalidad")
        return
    for p in datos["profesores"]:
        if p["nombre"] == prof:
            for a in p["asignaturas"]:
                if a["nombre"] == combo_asig.get():
                    for g in a["grupos"]:
                        if g["nombre"].lower() == nombre.lower():
                            messagebox.showwarning("Atención", "Grupo ya existe")
                            return
                    a["grupos"].append({"nombre": nombre, "modalidad": modalidad,
"estudiantes": []})
            agregado = True
            break
    if agregado:
        break

```

```

elif tipo == "estudiante":
    grupo = obtener_grupo()
    if grupo:
        for e in grupo["estudiantes"]:
            if e["nombre"].lower() == nombre.lower():
                messagebox.showwarning("Atención", "Estudiante ya existe")
                return
        grupo["estudiantes"].append({"nombre": nombre, "notas": [], "porcentajes":
[]})
        agregado = True

    if agregado:
        limpiar(entry)
        guardar()
        actualizar_vista()
        actualizar_combos()
        messagebox.showinfo("Éxito", f"{tipo.capitalize()} {nombre} agregado")

except Exception as e:
    messagebox.showerror("Error", f"Error al agregar {tipo}: {e}")

def eliminar(tipo):
    try:
        prof, asig, gru = combo_prof.get(), combo_asig.get(), combo_grupo.get()

        if tipo != "estudiante" and (not prof or (tipo != "profesor" and not asig) or (tipo ==
"grupo" and not gru)):
            messagebox.showwarning("Atención", "Seleccione el elemento a eliminar")
            return

        if not messagebox.askyesno("Confirmar", f"¿Eliminar {tipo}? Esta acción no se
puede deshacer."):
            return

        for i, p in enumerate(datos["profesores"]):
            if p["nombre"] == prof or tipo == "profesor":
                if tipo == "profesor":

```

```

        datos["profesores"].pop(i)
    else:
        for j, a in enumerate(p["asignaturas"]):
            if a["nombre"] == asig:
                if tipo == "asignatura":
                    p["asignaturas"].pop(j)
                else:
                    for k, g in enumerate(a["grupos"]):
                        if g["nombre"] == gru:
                            if tipo == "grupo":
                                a["grupos"].pop(k)
                            elif tipo == "estudiante":
                                nombre = entry_eliminar.get().strip()
                                if not nombre:
                                    messagebox.showwarning("Atención", "Escriba el nombre del
estudiante")

                                return
                            for l, e in enumerate(g["estudiantes"]):
                                if e["nombre"].lower() == nombre.lower():
                                    g["estudiantes"].pop(l)
                                    limpiar(entry_eliminar)
                                    messagebox.showinfo("Éxito", "Estudiante eliminado")
                                    break
                            break
                    break
            break
        break

    guardar()
    actualizar_vista()
    actualizar_combos()
    if tipo != "estudiante":
        messagebox.showinfo("Éxito", f"{tipo.capitalize()} eliminado")
except Exception as e:
    messagebox.showerror("Error", f"Error al eliminar {tipo}: {e}")

```

```

def eliminar_desde_lista():
    try:

```

```

sel = lista.curselection()
if not sel:
    messagebox.showwarning("Atención", "Seleccione un elemento")
    return
texto = lista.get(sel[0])
if "🏆" in texto:
    eliminar("profesor")
elif "📚" in texto:
    eliminar("asignatura")
elif "🏠" in texto or "💻" in texto:
    eliminar("grupo")
elif "🎓" in texto:
    nombre = texto.split("🎓 ")[1].split()[0]
    entry_eliminar.delete(0, tk.END)
    entry_eliminar.insert(0, nombre)
    eliminar("estudiante")
except Exception as e:
    messagebox.showerror("Error", f"Error al eliminar desde lista: {e}")

```

```

def definir_porcentajes():
    try:
        grupo = obtener_grupo()
        if not grupo or not grupo.get("estudiantes"):
            messagebox.showwarning("Atención", "No hay estudiantes en este grupo")
            return

```

```

v = tk.Toplevel()
v.title("Porcentajes")
centrar(v, 400, 450)
v.configure(bg=COLOR_FONDO)
vcmd = v.register(solo_numeros)

tk.Label(v, text=f"Grupo: {combo_grupo.get()}")
bg=COLOR_FONDO).pack(pady=5)

```

```

num = tk.IntVar(value=3)

```

```

tk.Label(v, text="¿Cuántas actividades?", bg=COLOR_FONDO).pack()
spin = tk.Spinbox(v, from_=1, to=6, textvariable=num, width=5)
spin.pack(pady=5)

frame = tk.Frame(v, bg=COLOR_FONDO)
frame.pack(pady=10)
entries = []

def actualizar():
    for w in frame.winfo_children():
        w.destroy()
    entries.clear()
    for i in range(num.get()):
        tk.Label(frame, text=f"Actividad {i + 1} (%)", bg=COLOR_FONDO).grid(row=i,
column=0, padx=5)
        e = tk.Entry(frame, width=10, validate="key", validatecommand=(vcmd, "%P"))
        e.grid(row=i, column=1, padx=5)
        entries.append(e)

actualizar()
spin.config(command=actualizar)

def guardar_pct():
    try:
        pcts = [float(e.get()) for e in entries]
        if sum(pcts) != 100:
            messagebox.showerror("Error", f"Suma {sum(pcts)}%, debe ser 100%")
            return
        for e in grupo["estudiantes"]:
            e["porcentajes"] = pcts.copy()
            e["notas"] = [0] * len(pcts)
        guardar()
        actualizar_vista()
        messagebox.showinfo("Éxito", f"Porcentajes: {pcts}")
        v.destroy()
    except:
        messagebox.showerror("Error", "Ingrese números válidos")

```

```

        tk.Button(v, text="Guardar", command=guardar_pct, bg=COLOR_VERDE,
fg="white").pack(pady=10)
    except Exception as e:
        messagebox.showerror("Error", f"Error al definir porcentajes: {e}")

def ingresar_notas():
    try:
        grupo = obtener_grupo()
        if not grupo or not grupo.get("estudiantes"):
            messagebox.showwarning("Atención", "No hay estudiantes")
            return
        for e in grupo["estudiantes"]:
            if not e.get("porcentajes"):
                messagebox.showwarning("Atención", "Primero defina porcentajes")
            return

    v = tk.Toplevel()
    v.title("Notas")
    centrar(v, 550, 500)
    v.configure(bg=COLOR_FONDO)
    vcmd = v.register(solo_numeros)

    tk.Label(v, text=f"Grupo: {combo_grupo.get()}", bg=COLOR_FONDO).pack()
    nb = ttk.Notebook(v)
    nb.pack(fill=tk.BOTH, expand=True, padx=10, pady=10)
    todas = {}

    for e in grupo["estudiantes"]:
        frame = tk.Frame(nb, bg=COLOR_FONDO)
        nb.add(frame, text=e["nombre"])
        todas[e["nombre"]] = []
        for i, pct in enumerate(e["porcentajes"]):
            f = tk.Frame(frame, bg=COLOR_FONDO)
            f.pack(pady=3)
            tk.Label(f, text=f"Actividad {i + 1} ({pct}%):",
bg=COLOR_FONDO).pack(side=tk.LEFT, padx=5)
            entry = tk.Entry(f, width=10, validate="key", validatecommand=(vcmd, "%P"))

```

```

        entry.pack(side=tk.LEFT)
        if e.get("notas") and i < len(e["notas"]) and e["notas"][i] != 0:
            entry.insert(0, str(e["notas"][i]))
        todas[e["nombre"]].append(entry)

def guardar_notas():
    try:
        for e in grupo["estudiantes"]:
            notas = []
            for entry in todas[e["nombre"]]:
                val = entry.get()
                notas.append(float(val) if val else 0)
            e["notas"] = notas
        guardar()
        actualizar_vista()
        messagebox.showinfo("Éxito", "Notas guardadas")
        v.destroy()
    except:
        messagebox.showerror("Error", "Ingrese números válidos")

tk.Button(v, text="Guardar todas", command=guardar_notas, bg=COLOR_VERDE,
fg="white").pack(pady=10)
except Exception as e:
    messagebox.showerror("Error", f"Error al ingresar notas: {e}")

def ver_reporte():
    try:
        texto = "=" * 50 + "\nREPORTE COMPLETO\n" + "=" * 50 + "\n\n"
        for p in datos.get("profesores", []):
            texto += f"\nPROFESOR: {p['nombre']}\n"
            for a in p.get("asignaturas", []):
                texto += f" Asignatura: {a['nombre']}\n"
                for g in a.get("grupos", []):
                    texto += f" Grupo: {g['nombre']}\n"
                    for e in g.get("estudiantes", []):
                        notas, pcts = e.get("notas", []), e.get("porcentajes", [])
                        if notas and pcts:

```



```

        final = sum(notas[i] * (pcts[i] / 100) for i in range(len(notas)))
        texto += f"    {e['nombre']}: {round(final, 2)} pts\n"
    else:
        texto += f"    {e['nombre']}: Sin notas\n"
v = tk.Toplevel()
v.title("Reporte")
centrar(v, 550, 500)
t = tk.Text(v, wrap=tk.WORD)
t.pack(fill=tk.BOTH, expand=True, padx=10, pady=10)
t.insert(tk.END, texto)
t.config(state=tk.DISABLED)
except Exception as e:
    messagebox.showerror("Error", f"Error al generar reporte: {e}")

```

```

def limpiar_datos_corruptos():
    global datos
    try:
        eliminados = 0
        for p in datos["profesores"][::]:
            if len(p["nombre"]) > 30:
                datos["profesores"].remove(p)
                eliminados += 1
            continue
        for a in p["asignaturas"][::]:
            if len(a["nombre"]) > 30:
                p["asignaturas"].remove(a)
                eliminados += 1
            continue
        for g in a["grupos"][::]:
            if len(g["nombre"]) > 30:
                a["grupos"].remove(g)
                eliminados += 1
            continue
        for e in g["estudiantes"][::]:
            if len(e["nombre"]) > 30:
                g["estudiantes"].remove(e)
                eliminados += 1
    
```

```

    guardar()
    actualizar_vista()
    actualizar_combos()
    messagebox.showinfo("Limpieza", f"Se eliminaron {eliminados} elementos con
más de 30 caracteres")
except Exception as e:
    messagebox.showerror("Error", f"Error al limpiar datos: {e}")

```

```

# ===== VENTANA PRINCIPAL DEL PROBLEMA 1
=====

```

```

def main():
    global entry_prof, entry_asig, entry_grupo, entry_est, entry_eliminar, combo_prof,
    combo_asig, combo_grupo, combo_mod, lista

```

```

    root = tk.Tk()
    root.title("Gestión de Notas")
    centrar(root, 1100, 700)
    root.configure(bg=COLOR_FONDO)
    root.resizable(False, False)

```

```

    vcmd_nombre = root.register(limitar_30)

```

```

# Menú
    menu = tk.Menu(root)
    root.config(menu=menu)
    m = tk.Menu(menu, tearoff=0)
    menu.add_cascade(label="Archivo", menu=m)
    m.add_command(label="Guardar", command=guardar)
    m.add_command(label="Cargar", command=cargar)
    m.add_separator()
    m.add_command(label="Limpiar Datos Corruptos",
command=limpiar_datos_corruptos)
    m.add_separator()
    m.add_command(label="Salir", command=root.quit)

```

```

# Panel izquierdo
    izq = tk.Frame(root, bg=COLOR_BLANCO, relief=tk.RIDGE, bd=1)

```

```

izq.pack(side=tk.LEFT, fill=tk.BOTH, expand=True, padx=10, pady=10)
tk.Label(izq, text="📁 ESTRUCTURA", font=("Arial", 11, "bold"),
bg=COLOR_BLANCO).pack(pady=5)
scroll = tk.Scrollbar(izq)
scroll.pack(side=tk.RIGHT, fill=tk.Y)
lista = tk.Listbox(izq, yscrollcommand=scroll.set, font=("Arial", 9))
lista.pack(fill=tk.BOTH, expand=True, padx=5, pady=5)
scroll.config(command=lista.yview)

# Panel derecho
der = tk.Frame(root, bg=COLOR_BLANCO, relief=tk.RIDGE, bd=1)
der.pack(side=tk.RIGHT, fill=tk.BOTH, expand=True, padx=10, pady=10)
nb = ttk.Notebook(der)
nb.pack(fill=tk.BOTH, expand=True, padx=10, pady=10)

# Pestaña DATOS
tab = tk.Frame(nb, bg=COLOR_BLANCO)
nb.add(tab, text="📄 DATOS")

tk.Label(tab, text="📌 SELECCIONAR", font=("Arial", 10, "bold"),
bg=COLOR_BLANCO).pack(pady=5)
f = tk.Frame(tab, bg=COLOR_BLANCO)
f.pack()
row = 0
tk.Label(f, text="Profesor:", bg=COLOR_BLANCO).grid(row=row, column=0, padx=5)
combo_prof = ttk.Combobox(f, width=30)
combo_prof.grid(row=row, column=1, padx=5)
combo_prof.bind("<<ComboboxSelected>>", lambda e: actualizar_combo_asig())

row = 1
tk.Label(f, text="Asignatura:", bg=COLOR_BLANCO).grid(row=row, column=0,
padx=5)
combo_asig = ttk.Combobox(f, width=30)
combo_asig.grid(row=row, column=1, padx=5)
combo_asig.bind("<<ComboboxSelected>>", lambda e: actualizar_combo_grupo())

row = 2
tk.Label(f, text="Grupo:", bg=COLOR_BLANCO).grid(row=row, column=0, padx=5)

```

```

combo_grupo = ttk.Combobox(f, width=30)
combo_grupo.grid(row=row, column=1, padx=5)

tk.Label(tab, text=" + AGREGAR", font=("Arial", 10, "bold"),
bg=COLOR_BLANCO).pack(pady=10)
f2 = tk.Frame(tab, bg=COLOR_BLANCO)
f2.pack()

row = 0
tk.Label(f2, text="Profesor:", bg=COLOR_BLANCO).grid(row=row, column=0,
padx=5)
entry_prof = tk.Entry(f2, width=30, validate="key",
validatecommand=(vcmd_nombre, "%P"))
entry_prof.grid(row=row, column=1, padx=5)
tk.Button(f2, text="Agregar", command=lambda: agregar(entry_prof, "profesor"),
bg=COLOR_AZUL, fg="white",
width=8).grid(row=row, column=2, padx=5)

row = 1
tk.Label(f2, text="Asignatura:", bg=COLOR_BLANCO).grid(row=row, column=0,
padx=5)
entry_asig = tk.Entry(f2, width=30, validate="key",
validatecommand=(vcmd_nombre, "%P"))
entry_asig.grid(row=row, column=1, padx=5)
tk.Button(f2, text="Agregar", command=lambda: agregar(entry_asig, "asignatura"),
bg=COLOR_AZUL, fg="white",
width=8).grid(row=row, column=2, padx=5)

row = 2
tk.Label(f2, text="Grupo:", bg=COLOR_BLANCO).grid(row=row, column=0, padx=5)
entry_grupo = tk.Entry(f2, width=18, validate="key",
validatecommand=(vcmd_nombre, "%P"))
entry_grupo.grid(row=row, column=1, padx=5)
combo_mod = ttk.Combobox(f2, values=["Presencial", "Distancia"], width=12)
combo_mod.grid(row=row, column=2, padx=5)
tk.Button(f2, text="Agregar", command=lambda: agregar(entry_grupo, "grupo"),
bg=COLOR_AZUL, fg="white",
width=8).grid(row=row, column=3, padx=5)

```

```

row = 3
tk.Label(f2, text="Estudiante:", bg=COLOR_BLANCO).grid(row=row, column=0,
padx=5)
entry_est = tk.Entry(f2, width=30, validate="key", validatecommand=(vcmd_nombre,
"%P"))
entry_est.grid(row=row, column=1, padx=5)
tk.Button(f2, text="Agregar", command=lambda: agregar(entry_est, "estudiante"),
bg=COLOR_AZUL, fg="white",
width=8).grid(row=row, column=2, padx=5)

tk.Label(tab, text="🗑 ELIMINAR", font=("Arial", 10, "bold"), bg=COLOR_BLANCO,
fg=COLOR_ROJO).pack(pady=10)
f3 = tk.Frame(tab, bg=COLOR_BLANCO)
f3.pack()
tk.Button(f3, text="Eliminar Profesor", command=lambda: eliminar("profesor"),
bg=COLOR_ROJO, fg="white",
width=15).grid(row=0, column=0, padx=5)
tk.Button(f3, text="Eliminar Asignatura", command=lambda: eliminar("asignatura"),
bg=COLOR_ROJO, fg="white",
width=15).grid(row=0, column=1, padx=5)
tk.Button(f3, text="Eliminar Grupo", command=lambda: eliminar("grupo"),
bg=COLOR_ROJO, fg="white", width=15).grid(
row=0, column=2, padx=5)

f4 = tk.Frame(tab, bg=COLOR_BLANCO)
f4.pack(pady=5)
tk.Label(f4, text="Nombre:", bg=COLOR_BLANCO).pack(side=tk.LEFT, padx=5)
entry_eliminar = tk.Entry(f4, width=25, validate="key",
validatecommand=(vcmd_nombre, "%P"))
entry_eliminar.pack(side=tk.LEFT, padx=5)
tk.Button(f4, text="Eliminar Estudiante", command=lambda: eliminar("estudiante"),
bg=COLOR_ROJO, fg="white",
width=18).pack(side=tk.LEFT, padx=5)

tk.Button(tab, text="🗑 Eliminar seleccionado", command=eliminar_desde_lista,
bg=COLOR_ROJO, fg="white",
padx=20).pack(pady=5)

```

```

# Pestaña CALIFICACIONES
tab2 = tk.Frame(nb, bg=COLOR_BLANCO)
nb.add(tab2, text="📊 CALIFICACIONES")
f5 = tk.Frame(tab2, bg=COLOR_BLANCO)
f5.pack(expand=True, pady=20)
tk.Button(f5, text="Definir Porcentajes", command=definir_porcentajes,
bg=COLOR_AZUL, fg="white", width=22,
pady=10).pack(pady=10)
tk.Button(f5, text="Ingresar Notas", command=ingresar_notas, bg=COLOR_AZUL,
fg="white", width=22, pady=10).pack(
pady=10)

# Pestaña REPORTES
tab3 = tk.Frame(nb, bg=COLOR_BLANCO)
nb.add(tab3, text="📁 REPORTES")
f6 = tk.Frame(tab3, bg=COLOR_BLANCO)
f6.pack(expand=True, pady=20)
tk.Button(f6, text="Ver Reporte", command=ver_reporte, bg=COLOR_AZUL,
fg="white", width=22, pady=10).pack(pady=10)
tk.Button(f6, text="Guardar", command=guardar, bg=COLOR_VERDE, fg="white",
width=22, pady=10).pack(pady=10)
tk.Button(f6, text="Cargar", command=cargar, bg=COLOR_AZUL, fg="white",
width=22, pady=10).pack(pady=10)

cargar()
actualizar_combos()
root.mainloop()

if __name__ == "__main__":
    main()

```

Problema 2

```
import tkinter as tk
from tkinter import ttk, messagebox, filedialog
import os
from cryptography.fernet import Fernet
from cryptography.hazmat.primitives.asymmetric import rsa, padding
from cryptography.hazmat.primitives import serialization, hashes
from cryptography.hazmat.backends import default_backend
```

```
# Colores (igual que problema1)
COLOR_FONDO = "#F0F0F0"
COLOR_BLANCO = "#FFFFFF"
COLOR_AZUL = "#0078D7"
COLOR_VERDE = "#388E3C"
COLOR_NARANJA = "#FF9800"
COLOR_MORADO = "#9C27B0"
```

```
def centrar(ventana, ancho, alto):
    ventana.update_idletasks()
    x = (ventana.winfo_screenwidth() - ancho) // 2
    y = (ventana.winfo_screenheight() - alto) // 2
    ventana.geometry(f"{ancho}x{alto}+{x}+{y}")
```

```
def seleccionar_entrada():
    archivo = filedialog.askopenfilename()
    if archivo:
        entry_entrada.delete(0, tk.END)
        entry_entrada.insert(0, archivo)
```

```
def seleccionar_salida():
    archivo = filedialog.asksaveasfilename()
    if archivo:
        entry_salida.delete(0, tk.END)
        entry_salida.insert(0, archivo)
```

```

def generar_clave():
    try:
        clave = Fernet.generate_key()
        entry_clave.delete(0, tk.END)
        entry_clave.insert(0, clave.decode())
        messagebox.showinfo("Éxito", "Clave generada")
    except Exception as e:
        messagebox.showerror("Error", str(e))

def generar_rsa():
    try:
        priv = rsa.generate_private_key(public_exponent=65537, key_size=2048)
        pub = priv.public_key()

        text_publica.delete(1.0, tk.END)
        text_publica.insert(1.0, pub.public_bytes(
            encoding=serialization.Encoding.PEM,
            format=serialization.PublicFormat.SubjectPublicKeyInfo
        ).decode())

        text_privada.delete(1.0, tk.END)
        text_privada.insert(1.0, priv.private_bytes(
            encoding=serialization.Encoding.PEM,
            format=serialization.PrivateFormat.PKCS8,
            encryption_algorithm=serialization.NoEncryption()
        ).decode())

        messagebox.showinfo("Éxito", "Llaves RSA generadas")
    except Exception as e:
        messagebox.showerror("Error", str(e))

def ejecutar():
    try:
        entrada = entry_entrada.get().strip()

```



```

salida = entry_salida.get().strip()

if not entrada or not salida:
    messagebox.showwarning("Atención", "Seleccione archivos")
    return

if metodo.get() == "simetrico":
    clave = entry_clave.get().strip()
    if not clave:
        messagebox.showwarning("Atención", "Ingrese una clave")
        return
    fernet = Fernet(clave.encode())

    if modo_encryptar.get():
        with open(entrada, 'rb') as f:
            datos = f.read()
        with open(salida, 'wb') as f:
            f.write(fernet.encrypt(datos))
    else:
        with open(entrada, 'rb') as f:
            datos = f.read()
        with open(salida, 'wb') as f:
            f.write(fernet.decrypt(datos))
    else:
        if modo_encryptar.get():
            pub = text_publica.get(1.0, tk.END).strip()
            if not pub:
                messagebox.showwarning("Atención", "Genere o cargue llave pública")
                return
            key = serialization.load_pem_public_key(pub.encode())
            with open(entrada, 'rb') as f:
                datos = f.read()
            encriptado = key.encrypt(
                datos,
                padding.OAEP(mgf=padding.MGF1(hashes.SHA256()),
algorithm=hashes.SHA256(), label=None)
            )
            with open(salida, 'wb') as f:

```

```

        f.write(encryptado)
    else:
        priv = text_privada.get(1.0, tk.END).strip()
        if not priv:
            messagebox.showwarning("Atención", "Genere o cargue llave privada")
            return
        key = serialization.load_pem_private_key(priv.encode(), password=None)
        with open(entrada, 'rb') as f:
            datos = f.read()
            desencryptado = key.decrypt(
                datos,
                padding.OAEP(mgf=padding.MGF1(hashes.SHA256()),
algorithm=hashes.SHA256(), label=None)
            )
        with open(salida, 'wb') as f:
            f.write(desencryptado)

    accion = "Encriptado" if modo_encriptar.get() else "Desencriptado"
    messagebox.showinfo("Éxito", f"Archivo {accion} correctamente")

except Exception as e:
    messagebox.showerror("Error", str(e))

def limpiar():
    entry_entrada.delete(0, tk.END)
    entry_salida.delete(0, tk.END)
    entry_clave.delete(0, tk.END)
    text_publica.delete(1.0, tk.END)
    text_privada.delete(1.0, tk.END)

def main():
    global entry_entrada, entry_salida, entry_clave, text_publica, text_privada
    global modo_encriptar, metodo

    root = tk.Tk()
    root.title("Problema 2 - Encriptación")

```

```

centrar(root, 700, 600)
root.configure(bg=COLOR_FONDO)
root.resizable(False, False)

# Título
tk.Label(root, text="🔒 ENCRIPCIÓN DE ARCHIVOS", font=("Arial", 14, "bold"),
        bg=COLOR_FONDO, fg=COLOR_MORADO).pack(pady=10)

# Marco
marco = tk.Frame(root, bg=COLOR_BLANCO, relief=tk.RIDGE, bd=1)
marco.pack(fill=tk.BOTH, expand=True, padx=15, pady=10)

# Modo
f_mod = tk.LabelFrame(marco, text="Modo", bg=COLOR_BLANCO)
f_mod.pack(fill=tk.X, padx=10, pady=5)
modo_encryptar = tk.BooleanVar(value=True)
tk.Radiobutton(f_mod, text="Encriptar", variable=modo_encryptar, value=True,
bg=COLOR_BLANCO).pack(side=tk.LEFT,
                                padx=20, pady=5)
tk.Radiobutton(f_mod, text="Desencriptar", variable=modo_encryptar,
value=False, bg=COLOR_BLANCO).pack(
    side=tk.LEFT, padx=20, pady=5)

# Método
f_metodo = tk.LabelFrame(marco, text="Método", bg=COLOR_BLANCO)
f_metodo.pack(fill=tk.X, padx=10, pady=5)
metodo = tk.StringVar(value="simetrico")
tk.Radiobutton(f_metodo, text="Simétrico", variable=metodo, value="simetrico",
bg=COLOR_BLANCO).pack(side=tk.LEFT,
                                padx=20,
                                pady=5)
tk.Radiobutton(f_metodo, text="Asimétrico (RSA)", variable=metodo,
value="asimetrico", bg=COLOR_BLANCO).pack(
    side=tk.LEFT, padx=20, pady=5)

# Archivos
f_archivos = tk.LabelFrame(marco, text="Archivos", bg=COLOR_BLANCO)
f_archivos.pack(fill=tk.X, padx=10, pady=5)

```

```

f1 = tk.Frame(f_archivos, bg=COLOR_BLANCO)
f1.pack(fill=tk.X, padx=10, pady=3)
tk.Label(f1, text="Entrada:", bg=COLOR_BLANCO, width=8).pack(side=tk.LEFT)
entry_entrada = tk.Entry(f1, width=45)
entry_entrada.pack(side=tk.LEFT, padx=5, fill=tk.X, expand=True)
tk.Button(f1, text="📁", command=seleccionar_entrada, bg=COLOR_AZUL,
fg="white", width=3).pack(side=tk.LEFT)

f2 = tk.Frame(f_archivos, bg=COLOR_BLANCO)
f2.pack(fill=tk.X, padx=10, pady=3)
tk.Label(f2, text="Salida:", bg=COLOR_BLANCO, width=8).pack(side=tk.LEFT)
entry_salida = tk.Entry(f2, width=45)
entry_salida.pack(side=tk.LEFT, padx=5, fill=tk.X, expand=True)
tk.Button(f2, text="📁", command=seleccionar_salida, bg=COLOR_AZUL,
fg="white", width=3).pack(side=tk.LEFT)

# Simétrico
f_simetrico = tk.LabelFrame(marco, text="Clave Simétrica", bg=COLOR_BLANCO)

f3 = tk.Frame(f_simetrico, bg=COLOR_BLANCO)
f3.pack(fill=tk.X, padx=10, pady=5)
tk.Label(f3, text="Clave:", bg=COLOR_BLANCO).pack(side=tk.LEFT)
entry_clave = tk.Entry(f3, width=50)
entry_clave.pack(side=tk.LEFT, padx=5, fill=tk.X, expand=True)

f4 = tk.Frame(f_simetrico, bg=COLOR_BLANCO)
f4.pack(pady=5)
tk.Button(f4, text="Generar", command=generar_clave, bg=COLOR_NARANJA,
fg="white", padx=10).pack(side=tk.LEFT,
                                padx=5)

# Asimétrico
f_asimetrico = tk.LabelFrame(marco, text="Claves RSA", bg=COLOR_BLANCO)

f5 = tk.Frame(f_asimetrico, bg=COLOR_BLANCO)
f5.pack(fill=tk.X, padx=10, pady=5)
tk.Label(f5, text="Pública:", bg=COLOR_BLANCO).pack(anchor="w")

```

```

text_publica = tk.Text(f5, height=4, width=70)
text_publica.pack(pady=3, fill=tk.X)

f6 = tk.Frame(f_asimetrico, bg=COLOR_BLANCO)
f6.pack(fill=tk.X, padx=10, pady=5)
tk.Label(f6, text="Privada:", bg=COLOR_BLANCO).pack(anchor="w")
text_privada = tk.Text(f6, height=4, width=70)
text_privada.pack(pady=3, fill=tk.X)

f7 = tk.Frame(f_asimetrico, bg=COLOR_BLANCO)
f7.pack(pady=5)
tk.Button(f7, text="Generar RSA", command=generar_rsa, bg=COLOR_NARANJA,
fg="white", padx=10).pack(side=tk.LEFT,
                                padx=5)

# Mostrar/ocultar según método
def actualizar_metodo():
    if metodo.get() == "simetrico":
        f_simetrico.pack(fill=tk.X, padx=10, pady=5)
        f_asimetrico.pack_forget()
    else:
        f_simetrico.pack_forget()
        f_asimetrico.pack(fill=tk.X, padx=10, pady=5)

# Botones
btn_ejecutar = tk.Button(marco, text="▶ Ejecutar", command=ejecutar,
                        bg=COLOR_VERDE, fg="white", font=("Arial", 11, "bold"), padx=20,
pady=8)
btn_ejecutar.pack(pady=10)

tk.Button(marco, text="Limpiar", command=limpiar, bg="gray", fg="white",
padx=20).pack(pady=5)
tk.Button(marco, text="Cerrar", command=root.destroy, bg="gray", fg="white",
padx=20).pack(pady=5)

# Eventos
metodo.trace_add("write", lambda *_: actualizar_metodo())
actualizar_metodo()

```

```
root.mainloop()
```

```
if __name__ == "__main__":  
    main()
```